

Analiza și Testarea Aplicației Tax Calculator

Proiect TSS 2026

Stefan Tacu

15 mai 2026

1. Introducere și Configurație

- **Obiectiv:** Testarea riguroasă a unei aplicații de calcul fiscal.
- **Stack Tehnic:**
 - Python 3.13.2, Pytest 8.1.1
 - Radon (Complexity), Mutmut (Mutation)
 - Coverage.py (Structural Coverage)
- **Funcționalitate:** Calcul taxe pe categorii (Salary, Business, Crypto, etc.) cu aplicare de deduceri complexe.

2. Testare Funcțională (Black-Box)

- **Tehnici:** Equivalence Partitioning & Boundary Value Analysis.
- **Clase de Echivalență:**
 - Venit: Valid $[0, 1M]$, Invalid $(j0, i1M)$.
 - Vârstă: Valid $[0, 150]$, Invalid.
 - Categori: 6 categorii valide + ramură default.
- **Status:** 100% teste funcționale trecute.

3. Analiza Structurală: Teoria McCabe

- **Complexitate Ciclomatică (CC):** Măsură a numărului de circuite liniar independente.
- **Formula McCabe:** $V(G) = e - n + 2p$ (unde $p = 1$ pentru o subrutină).
- **Aplicație:** În proiectul nostru, $V(G) = 41$ (Grad F).
- **Testarea Circuitelor Independente:**
 - Identifică limita superioară pentru numărul de căi necesare acoperirii ramurilor.
 - **Set de bază:** Orice cale prin program se poate forma ca o combinație din acest set.
- **Avantaj:** Setul poate fi generat pentru a garanta *Branch Coverage* 100%.

4. Rezultate Acoperire Structurală

The screenshot shows the VS Code interface with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'TIS Project 2026' with folders like 'App', 'docs', 'build', 'capturi_ecran', 'statement_coverage', 'statement_coverage_reports', 'diagrame', 'model_profesoara', 'structural_coverage_reports', and 'branch_coverage_branch'. The code editor displays the file 'Tax_Calculator.py' with a coverage report at the top: 'Coverage for App / Tax_Calculator.py: 100.00%'. Below this, it shows '66 statements' with '66 run', '0 missing', '1 excluded', and '0 partial'. The code itself is a Python class 'TaxEngine' with methods 'calculate_annual_tax' and 'validate_date'. The coverage is indicated by green bars on the left side of the code lines.

```
from typing import Union

class TaxEngine:
    def __init__(self):
        self.min_income = 0
        self.max_income = 1000000
        self.min_age = 0
        self.max_age = 150
        self.valid_categories = {
            "salary",
            "business",
            "investment",
            "freelance",
            "crypto",
            "real_estate",
        }

    def calculate_annual_tax(
        self,
        income: Union[int, float],
        category: str,
        age: int,
        is_resident: bool,
        has_dependents: bool = False,
        is_married: bool = False,
    ):
        # 1. Validare date (Clase de echivalenta)
        if not isinstance(income, (int, float)) or not isinstance(age, int):
            return "Error: Invalid Data Type"

        if income < self.min_income or income > self.max_income:
            return "Error: Invalid Income"
```

- **Statement Coverage: 100%**
- **Branch Coverage: 100%** (obținut după rafinarea suitei de teste pentru a acoperi ramurile else implicite).

5. Testarea prin Mutație (Mutation Testing)

- **Instrument:** mutmut (mutanți de ordin 1).
- **Rezultate Automate:**
 - Total mutanți: 228
 - Omorâți: 181
 - **Mutation Score: 79.3%**
- **Analiză Manuală:** Scor de 72.7% pe un eșantion reprezentativ de 22 de mutanți.

6. Analiza Mutanților: Supraviețuitori

- **Mutanți Echivalenți:** Modificarea $>$ în $\geq$ la praguri unde diferența este zero (ex: Business category).
- **Lipsă Precizie:** Supraviețuirea la modificarea rotunjirii (`round(tax, 1)`) a indicat aserțiuni prea permissive.
- **Îmbunătățire:** Rezultatele au condus la adăugarea de teste de frontieră mai stricte pentru categoriile Investment și Freelance.

7. Utilizare AI: GitHub Copilot

- **Eficiență:** Creștere cu 40% a vitezei de scriere a testelor de tip boiler-plate.
- **Puncte Forte:** Sugestii rapide pentru clasele de echivalență standard.
- **Puncte Slabe:** Riscul de "halucinații" la calcule matematice complexe și omiterea cazurilor de frontieră pe logica de CC 41.
- **Concluzie:** AI-ul este un copilot, dar decizia de testare rămâne la inginer.

8. Concluzii

- **Rigoare:** Branch Coverage de 100% este necesar dar nu suficient fără Mutation Testing.
- **Complexitate:** Codul cu CC 41 (Rank F) este greu de întreținut și necesită teste automate dense.
- **Validare:** Integrarea metodelor manuale cu cele automate oferă cel mai înalt grad de încredere în software.

Vă mulțumesc!

Întrebări și discuții.